

DevOps Bootcamp

Tutorial-Hell Recovery Edition

A 4-6 week, AWS-free-tier curriculum that takes you from “I watched the course” to “I built and operated it.”

Cloud: AWS (us-east-1)

Cluster: k3s on EC2 t3.micro

GitOps: ArgoCD

Observability: Prometheus, Grafana, Loki

IaC: Terraform + Helm

Cost (with discipline): \$0/month

Open `pdf/guide.pdf` on a tablet.

Work in the repo on your laptop.

Paste `AGENT_SYSTEM_PROMPT.md` into your AI before every session.

Table of Contents

How to use this guide	3
The deal with the AI	3
The deal with money	3
The architecture you'll end up with	4
One more thing	4
Module 0 — Linux Fundamentals Refresher	5
Why this module exists	5
Concepts	5
What you'll do	5
Validation	6
Module 1 — AWS Foundations + Terraform	7
Why this module exists	7
Concepts	7
What you'll do	8
Validation	8
Module 2 — k3s Cluster on EC2	9
Why k3s instead of EKS	9
Concepts	9
What you'll do	9
Validation	10
Module 3 — Helm Packaging	11
Why Helm	11
Concepts	11
What you'll do	11
Validation	12
Module 4 — ArgoCD + GitOps	13
Why GitOps	13
Concepts	13
What you'll do	14
Validation	14
Module 5 — Monitoring (Prometheus + Grafana)	15
Why monitoring	15
Concepts	15
What you'll do	15
Validation	16
Module 6 — Logging + Capstone	17
Phase A: Logging	17
Phase B: The Capstone	17
Closing: the exit checklist	19

How to use this guide

READ FIRST

This is the **primary** curriculum. The repo's `modules/NN-*/` directories are companions: they hold the **exercises**, the **validation checklists**, and the per-module AI guardrails. The PDF teaches; the repo is where you do.

You picked up this guide because you've finished N tutorials and remember M of them, where M is alarmingly close to zero. Tutorials teach **recognition**, not **recall**. Recall comes from building and breaking your own systems. This is a recall machine.

The structure:

#	Module	Weeks
0	Linux fundamentals refresher	0.5
1	AWS foundations + Terraform	1.0
2	k3s cluster on EC2	0.5
3	Helm packaging	1.0
4	ArgoCD + GitOps	1.0
5	Monitoring (Prometheus + Grafana)	1.0
6	Logging + capstone debug scenario	1.0

The deal with the AI

This bootcamp assumes you'll use an AI assistant. It also assumes that letting the AI write your Terraform and YAML will produce the same outcome as the last 12 tutorials: you'll feel productive and remember nothing.

So the AI is **constrained**. Before each session, paste `AGENT_SYSTEM_PROMPT.md` into your AI. It will:

- Ask questions before answering.
- Point you at docs instead of writing code.
- Escalate hints only when you ask again.
- Refuse to write code until you explicitly type **"I'm stuck, draft it."**

AI GUARDRAIL

The AI's job is to make you faster **at thinking**, not faster at producing artifacts. Every line of code in this repo must be typed by you.

The deal with money

Everything in this bootcamp fits inside the AWS free tier **if you stay disciplined**. The main risk is leaving things running.

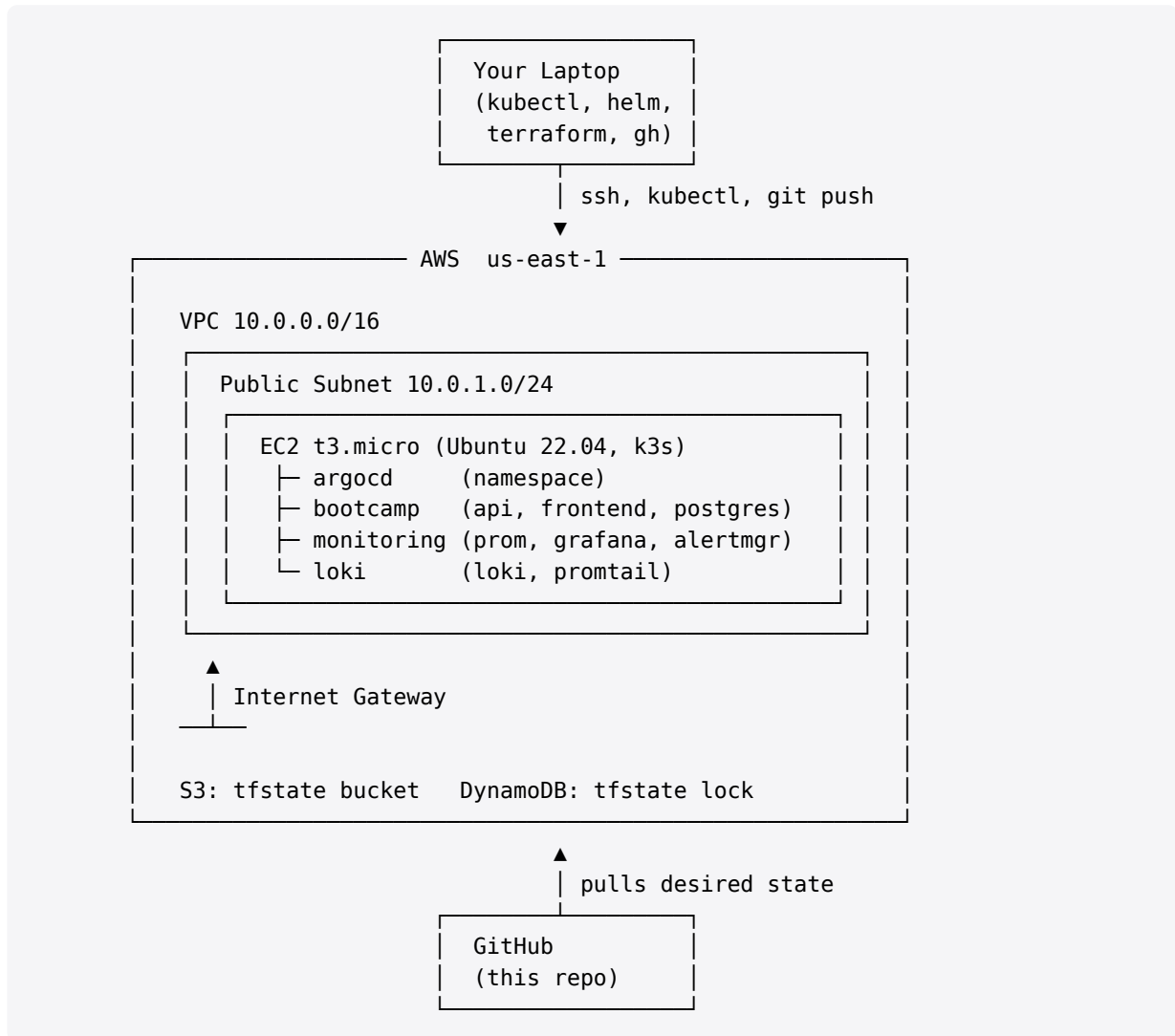
COST NOTE

End every session with `./scripts/check-free-tier.sh`.

Walk away for > 48 hours? `./scripts/destroy-all.sh`.

Set a \$5/mo budget alarm on day one. Walk-through in `docs/aws-account-setup.md`.

The architecture you'll end up with



One more thing

Do the modules in order. Module 4 (ArgoCD) assumes you understand Helm from Module 3. Module 5 (monitoring) assumes you understand Module 4's GitOps loop. Skipping ahead because something looks more fun is how you ended up here in the first place.

Module 0 — Linux Fundamentals Refresher

Time: ~0.5 week (4–6 hours)

Free-tier risk: None (entirely local)

Repo dir: modules/00-linux/

Why this module exists

You will spend the rest of the bootcamp logging into a Linux box, reading logs, and debugging systemd units. If your fingers don't already know `journalctl`, `ss`, and `systemctl status` by reflex, every later module triples in difficulty.

This is not “learn Linux.” It's “rebuild the muscle for the seven commands you'll use 200 times in the next 6 weeks.”

Concepts

systemd in one paragraph

systemd is the init system on every distro you'll touch. Each long-running service is described by a **unit file** (`.service`, `.timer`, etc.). You start, stop, enable (boot-time), and inspect units with `systemctl`. Each unit's stdout/stderr goes to `journald`, queryable with `journalctl`.

The seven commands

<code>systemctl status <unit></code>	Is it running? Why or why not?
<code>systemctl restart <unit></code>	Stop and start.
<code>systemctl enable --now <unit></code>	Start it, and on every boot.
<code>journalctl -u <unit> -n 100</code>	Last 100 lines from this unit.
<code>journalctl -u <unit> -p err</code>	Only errors from this unit.
<code>ss -tlnp</code>	What's listening, by which process.
<code>ps auxf</code>	Process tree with details.

What you'll do

A condensed view of `modules/00-linux/exercises.md`:

1. Inspect your Linux VM (OS, RAM, CPU, listening ports).
2. Drive `systemctl` and `journalctl` on the `sshd` service.
3. Write a `hellogick.service` unit that runs a script every 10 seconds for 5 minutes.
4. Deliberately break the unit (bad `ExecStart`). Use `journalctl` to find the cause.
5. Networking: `curl -v`, `dig`, `ss -tlnp`, run a one-liner Python HTTP server.
6. SSH ergonomics: `keypair`, `~/.ssh/config` aliases, `scp/rsync`.

READ THE DOCS

systemd: systemd.io · Arch wiki on systemd: wiki.archlinux.org/title/Systemd (best per-feature reference even on non-Arch)

AI GUARDRAIL

Module 0 AI rule: the AI may **not** write commands for you in this module. Even on “I’m stuck, draft it” the most it produces is a skeleton with **# fill this in** placeholders. The whole point is finger memory.

Validation

You’re done when, without notes, you can:

- Tell me whether **sshd** is running and enabled, with one command each.
- Show only the errors from **sshd** since boot in one command.
- List every listening port and the process that opened it.
- Write a systemd unit from a blank file with **Restart=on-failure** and a **network-online.target** dependency.

Module 1 — AWS Foundations + Terraform

Time: ~1 week (10–15 hours)
Free-tier risk: Low if you stay on t3.micro and tear down
Repo dir: infra/terraform/

Why this module exists

You will provision a small AWS footprint with hand-written Terraform. Not via console, not via someone's giant module. You will know exactly what every resource is and why it's there. The next four modules sit on top of this footprint.

Concepts

The Terraform loop

write HCL → terraform init → terraform plan → read plan → terraform apply → state updated

Repeat. The **plan** is the only meaningful sanity check between you and unintended changes. Reading it is a skill — the diff syntax (+, -, ~) tells you create / destroy / in-place modify. A **replace** (-/+) means the resource is destroyed and recreated — sometimes this is fine, sometimes catastrophic.

Remote state

Local state (`terraform.tfstate`) is fine for solo learning but breaks the moment a teammate joins. The standard pattern is **S3 + DynamoDB**: S3 holds the state file (versioned, encrypted), DynamoDB holds a lock so two `applies` can't race.

There's a chicken-and-egg problem: the bucket that holds your state can't itself be created by the Terraform that uses the bucket. Solution: a small bootstrap script (`scripts/bootstrap-tf-backend.sh`) creates the bucket + table by hand, once.

The minimum viable AWS footprint

```
VPC (10.0.0.0/16)
├─ Public Subnet (10.0.1.0/24, us-east-1a)
│   └─ EC2 t3.micro
│       ├── Security Group: 22, 6443 from your IP only
│       └─ IAM Instance Profile: read-only access to tfstate bucket
├─ Internet Gateway
└─ Route Table: 0.0.0.0/0 → IGW
```

That's **seven** primary resources plus their associations. Nothing else.

COST NOTE

Avoid: NAT Gateway (\$32/mo), ALB/NLB (\$22+/mo), EKS (\$72/mo), RDS (easy to misconfigure into a charge), Route 53 hosted zone (\$0.50/mo). See `docs/free-tier-budget.md` for the full list.

What you'll do

1. Configure AWS CLI with a **bootcamp** profile. Set a \$5 budget alarm.
2. Bootstrap state backend (S3 + DynamoDB) via the helper script.
3. Write **versions.tf**, **providers.tf**, **backend.tf**.
4. Write the VPC + subnet + IGW + route table by hand.
5. Write the security group + key pair + EC2 instance.
6. Write the IAM role + instance profile.
7. Refactor into modules: **vpc/**, **ec2/**, **iam/**.
8. Destroy. Re-apply. Confirm idempotency.

WARNING

Do **not** `terraform apply -auto-approve` while learning. Read every line of the plan. The day you stop reading plans is the day you delete your prod VPC.

READ THE DOCS

Terraform intro: hashicorp.com/terraform/intro · VPC concepts: docs.aws.amazon.com/vpc ·

Remote state: [terraform/language/state/remote](https://terraform.io/language/state/remote)

Validation

`terraform plan` on a freshly-applied stack returns “No changes.” `terraform state list` shows every resource and you can explain each. You can SSH to the EC2 via a config alias. `aws sts get-caller-identity` from inside the **EC2** returns the IAM role.

Module 2 — k3s Cluster on EC2

Time: ~0.5 week

Free-tier risk: Low — same EC2 as Module 1

Repo dir: infra/terraform/ (cloud-init) + remote ops on the EC2

Why k3s instead of EKS

EKS control plane bills you \$0.10/hour, about \$72/month. Not free. k3s is a single-binary, production-grade Kubernetes distribution (used at the edge, in CI, in IoT). It runs comfortably on a t3.micro at our scale and you'll learn **real** Kubernetes — Deployments, Services, Pods, ConfigMaps, NetworkPolicies — without the bill.

In an interview you can say: “I built and operated a k3s cluster from bare cloud. I know what the control plane components do because I started them.”

Concepts

What k3s ships with

```
k3s server (single binary)
├─ kube-apiserver
├─ kube-controller-manager
├─ kube-scheduler
├─ kubelet (embedded)
├─ containerd (embedded)
├─ CoreDNS                ← keep
├─ traefik (ingress)       ← disable for bootcamp
├─ servicelb (klipper)     ← disable (no real LB on free tier)
├─ local-path provisioner ← keep (storage class for Postgres)
└─ metrics-server         ← keep (kubectl top)
```

The flow

Terraform provisions the EC2 with cloud-init user data → cloud-init installs k3s on first boot → k3s writes kubeconfig to /etc/rancher/k3s/k3s.yaml → you SCP it down, rewrite **server:** to the public IP, point KUBECONFIG at it from your laptop.

What you'll do

1. Modify EC2's cloud-init to install k3s (pinned version) with `--disable servicelb,traefik` and `--tls-san <public-ip>`.
2. `terraform apply` (forces EC2 re-creation if it already existed).
3. SCP kubeconfig down, fix **server:** field, set KUBECONFIG.
4. `kubectl get nodes` from laptop — Ready.
5. Write a scratch `nginx-test.yaml` (Deployment + NodePort Service) by hand. Apply, browse to the node port, delete.
6. Break things deliberately: bad image tag, oversize resource request, NetworkPolicy blocking egress.

AI GUARDRAIL

When the engineer reports a workload issue, your AI is configured to first ask: “what does `kubectl describe` say? what about `kubectl get events --sort-by=.lastTimestamp`?” If you haven’t run those, that’s the answer.

READ THE DOCS

k3s: docs.k3s.io · K8s Deployments: [k8s/Deployment](https://k8s.io/docs/concepts/workloads/controllers/deployment) · K8s Services: [k8s/Service](https://k8s.io/docs/concepts/workloads/controllers/service)

Validation

`kubectl get nodes` from laptop returns `Ready`. `kubectl get pods -A` shows only system pods after you’ve cleaned up. You can destroy + recreate the EC2 via Terraform and end up with a working cluster (same cloud-init).

Module 3 — Helm Packaging

Time: ~1 week
Free-tier risk: None
Repo dir: infra/helm/

Why Helm

You have raw YAML. You've felt the pain of copy-pasting between dev and staging and producing drift. Helm fixes it with templates and values: one source of truth, many environments, idempotent installs.

Concepts

A chart is...

```
api/
├── Chart.yaml           ← name, version, appVersion, dependencies
├── values.yaml          ← defaults
├── values-dev.yaml      ← env-specific overrides
├── templates/
│   ├── _helpers.tpl     ← named templates (e.g., labels block)
│   ├── deployment.yaml  ← Go-template ⇒ K8s YAML
│   ├── service.yaml
│   └── configmap.yaml
└── tests/               ← `helm test` runs these as Jobs
```

`helm template` evaluates the templates against the values and prints the resulting YAML. Use it constantly while developing.

The install loop

`helm lint` → `helm template` → `helm install --dry-run` → `helm upgrade --install` → `helm history` → `helm rollback` (if needed)

`helm upgrade --install` is idempotent — use it instead of `helm install` so your script can be re-run.

Subcharts and umbrellas

An umbrella chart depends on other charts (local via `file://` or remote via a Helm repo). Values for subcharts live under a key named after the subchart. This is how you'll bundle api + frontend + postgres into one release.

What you'll do

1. Build & push images for `app/api` and `app/frontend` (Docker Hub is fine for the bootcamp).
2. Hand-write a chart for the API: Deployment, Service, ConfigMap, Secret reference.
3. Install via `helm upgrade --install` into a `bootcamp` namespace. Install Postgres via the Bitnami chart.
4. Repeat for the frontend.
5. Build an umbrella chart that depends on both + Postgres.
6. Create `values-dev.yaml` and `values-staging.yaml` with different `replicaCount` and `resources`.
7. Test a rollback: deploy a bad image tag, `helm rollback`, watch recovery.

WARNING

Never tag images **latest**. Always use a concrete tag (e.g., **v1.0.0**). Latest breaks rollback and reproducibility — you can't roll back to a specific build if every deploy points at “whatever was latest at that moment.”

READ THE DOCS

Helm charts: helm.sh/docs/topics/charts · Template guide: helm.sh/docs/chart_template_guide

Validation

`helm lint` clean on all charts. From a fresh `kubectl delete namespace bootcamp`, `helm upgrade --install` produces a working app in 3 minutes. `helm template` outputs have no unresolved `{{ ... }}`. The frontend in a browser lists items from the API.

Module 4 — ArgoCD + GitOps

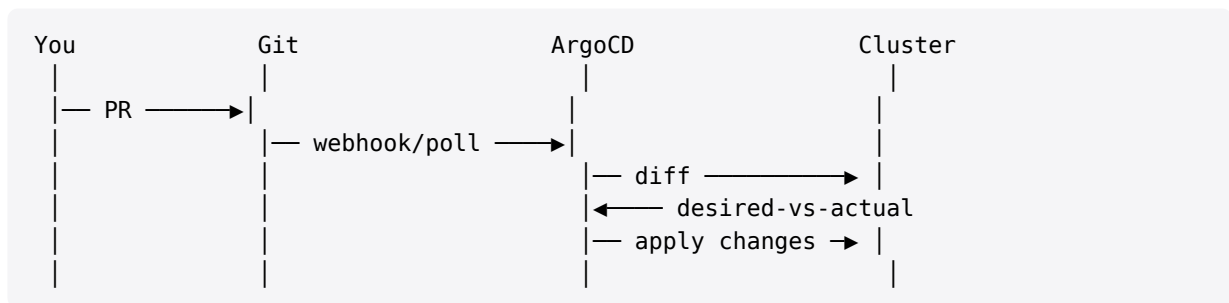
Time: ~1 week
Free-tier risk: None
Repo dir: infra/argocd/

Why GitOps

Push-from-laptop is fine for solo learning. It breaks at the first teammate. GitOps inverts the direction: **git is the source of truth, the cluster pulls**. Auditable. Reproducible. Roll-back-able with `git revert`.

Concepts

The reconciliation loop



ArgoCD continuously diffs Git’s desired state against the cluster’s actual state. When they diverge, it shows `OutOfSync`. You sync (manual) or it auto-syncs (`syncPolicy.automated`).

App-of-apps

One root **Application** points at a directory of **more Application** manifests. ArgoCD creates those child **Applications**, which create the actual workloads. Why: one bootstrap command unfolds your entire platform.

```

root (Application)
├── api           (Application → infra/helm/api)
├── frontend      (Application → infra/helm/frontend)
├── postgres      (Application → bitnami/postgresql)
├── monitoring    (Application → kube-prometheus-stack)
└── loki          (Application → grafana/loki-stack)
  
```

The three flags everyone gets wrong on day one

<code>automated.prune</code>	Delete resources in cluster when removed from Git. Powerful and dangerous — enable only after the App is stable.
<code>automated.selfHeal</code>	Revert manual <code>kubectl</code> edits in the cluster. Safe-ish; turn on early.
<code>syncOptions: [CreateNamespace=true]</code>	Lets the sync create the destination namespace. Always pair with <code>destination.namespace</code> .

What you'll do

1. Install ArgoCD via Helm (NodePort, no LB, no Ingress yet).
2. Get initial admin password, rotate it.
3. Bootstrap root Application by hand.
4. Create child Applications for `api`, `frontend`, `postgres`.
5. Delete your prior `helm` installs — ArgoCD is the new owner.
6. Drive a change via PR. Watch sync.
7. Manually `kubectl edit` the cluster. Watch ArgoCD detect drift. Sync to recover.
8. Enable `selfHeal: true`. Watch it auto-revert.
9. Enable `prune: true`. Remove a Service from Git, watch it delete in the cluster.

WARNING

Do **not** enable `prune` and `selfHeal` together on day one. Start manual. Promote to `selfHeal`. Then add `prune` once you've watched drift behavior with your own eyes.

READ THE DOCS

ArgoCD getting started: argo-cd.readthedocs.io/getting_started · App-of-apps: argo-cd/cluster-bootstrapping · GitOps principles: opengitops.dev

Validation

`argocd app list` shows root + 3 children, all Synced and Healthy. A merged PR produces a sync without you. Manual drift is auto-reverted. No `helm` releases of the bootcamp app exist outside ArgoCD.

Module 5 — Monitoring (Prometheus + Grafana)

Time: ~1 week

Free-tier risk: Low (EBS for PVCs — keep small)

Repo dir: observability/prometheus/, observability/grafana/

Why monitoring

You shipped. You don't know if it's working. Monitoring fixes that.

Concepts

The four golden signals (Google SRE book)

Signal	What	Sample PromQL
Traffic	Requests per second	<code>sum(rate(http_request_duration_seconds_count[5m]</code>
Errors	Error rate or ratio	<code>sum(rate(...{status=~"5.."}[5m])) / sum(rate(...[5m]))</code>
Latency	p50 / p95 / p99	<code>histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m] by (le))</code>
Saturation	How “full” the system is	<code>container_cpu_usage_seconds_total / cpu_quota</code>

The Prometheus Operator's CRDs

ServiceMonitor	Tells Prometheus to scrape a Service. Selector matches Service labels.
PodMonitor	Same but for Pods without a Service.
PrometheusRule	Recording rules and alerting rules.
Alertmanager	Top-level Alertmanager config.

You install all of this with one chart: `kube-prometheus-stack`.

What `rate()` actually computes

`rate(counter[5m])` = average increase per second over the last 5 minutes. The counter **must** be monotonically increasing (resets are handled internally). For dashboards: use `rate()` with a window $2-4 \times$ your scrape interval. For sub-second analysis: don't, sample more often instead.

What you'll do

1. Add `kube-prometheus-stack` as a new ArgoCD Application.
2. Constrain values: NodePort (no LB), 5 GiB Prometheus storage, 5-day retention.
3. Write a `ServiceMonitor` template in the api chart, guarded by `.Values.metrics.enabled`.
4. Verify the API target shows UP in Prometheus Targets page.
5. Run a small traffic generator in the background to fill dashboards.
6. Build a custom dashboard: 5 panels covering traffic, p50/p95/p99 latency, error rate, CPU, memory.

7. Export as JSON, store in `observability/grafana/dashboards/`, provision via ConfigMap so it survives Grafana restarts.
8. Write a `PrometheusRule` for `APIHighErrorRate`. Make it fire by scaling Postgres to 0. Recover.

AI GUARDRAIL

The AI in this module will not draft Grafana dashboard JSON from scratch — the schema is too large to do safely. It will instead point you at the JSON Model view of a community dashboard and ask you to adapt.

READ THE DOCS

Prometheus practices: prometheus.io/practices/instrumentation · Prometheus Operator: prometheus-operator.dev · SRE Monitoring chapter: sre.google/sre-book

Validation

Prometheus Targets shows your API UP. Custom dashboard renders with real moving data. Dashboard JSON in git survives a Grafana pod delete. `APIHighErrorRate` alert can be made to fire and clear by your own action.

Module 6 — Logging + Capstone

Time: ~1 week

Free-tier risk: Low (Loki PVC)

Repo dir: observability/loki/, modules/06-logging-capstone/

This module is two distinct phases: a logging-stack setup, then a **capstone** debugging exercise — the most important hour of the bootcamp.

Phase A: Logging

Concepts

Loki is “Prometheus for logs”: indexes by **labels**, not by full-text. This makes it cheap and fast for the common case (filter by namespace/pod/job, then grep). It makes high-cardinality fields (like user IDs as labels) expensive — keep them in the log **body**, not in labels.

LogQL is to Loki what PromQL is to Prometheus:

```
{namespace="bootcamp", pod=~"api.*"} |~ "(?i)error"
    └── label selector ─┘      └── line filter ─┘

sum by (pod) (count_over_time({namespace="bootcamp"} |= "ERROR" [5m]))
```

What you’ll do

1. Install **loki-stack** (loki + promtail) via ArgoCD.
2. Provision Grafana data source for Loki via ConfigMap.
3. Practice LogQL in Grafana Explore.
4. Build a row of panels in your Module 5 dashboard: top = error rate metric, bottom = filtered API logs.

Phase B: The Capstone

You run `./modules/06-logging-capstone/break-it.sh`. It picks one of five scenarios at random and breaks the system. You diagnose using only the dashboards and **kubectl**.

The rules

1. Write **hypothesis.md** **before** attempting any fix. The AI refuses to give problem-specific hints until your hypothesis is committed.
2. No **kubectl edit** to “see if it fixes it” — every change is preceded by a description of what you expect it to do.
3. Don’t read **break-it.sh**’s source until you’ve fixed it. The AI is also instructed not to read it.
4. Time the exercise. Typical: 1–3 hours.
5. After the fix, write **resolution.md**: root cause, breadcrumb trail, what you’d add to catch this in 30 seconds next time.

WARNING

This is the only module where the AI’s hint policy is **more** restrictive than your default. It is doing this on purpose. The capstone is the closest thing to a 2am page you can simulate without one.

The scenario pool

The break script picks one of these (your AI knows the categories but not the specifics):

Configuration	A Secret/ConfigMap value is wrong.
Image	Wrong image, wrong tag, image-pull failure.
Network	A NetworkPolicy or DNS issue blocks traffic.
Capacity	Resource limits too tight, replicas too low, node pressure.
Data layer	Database is down, unreachable, or the schema is gone.

Done-when

Frontend renders a non-empty list. `kubectl get all -n bootcamp` clean. Dashboard back to baseline. `argocd app list` Synced. `resolution.md` committed.

Closing: the exit checklist

You've finished the bootcamp when you can answer YES, without notes, to each of these:

1. I can **terraform apply** a VPC + EC2 from scratch and explain every resource on the plan.
2. I can SSH to the node, inspect a misbehaving systemd service, and find the cause in logs.
3. I can **helm install** my app from clean and explain what every value controls.
4. I can break ArgoCD sync intentionally and diagnose drift via the UI and CLI.
5. I can write a PromQL query that distinguishes "API is slow" from "API is down."
6. I can grep Loki for a request's logs and tell the story end-to-end.
7. I can destroy everything with one command and confirm \$0 ongoing spend.

If all seven are yes: you're out of tutorial hell. You're an apprentice operator.

If any one is shaky: that's where you go back. The modules don't expire.

Built for the engineer who watched the courses and wants to actually remember.

Repo: [this directory](#). PDF: [pdf/guide.pdf](#). AI prompt: [AGENT_SYSTEM_PROMPT.md](#).